

# Accounting for Trust When Machines Write Proofs

Halley Young  
Microsoft Research  
halleyyoung@microsoft.com

March 22, 2026

## Abstract

Modern verification pipelines combine evidence from radically heterogeneous sources: SMT solvers return **unsat**, large-language models declare “looks correct,” runtime monitors observe 95% test-pass rates, and human reviewers attest correctness by inspection. No existing system tracks the *provenance* of each piece of evidence or prevents the silent upgrading of a weak claim to a strong one. We introduce the *trust ordered algebra*  $\mathfrak{T} = (\mathcal{E}_{\text{adm}}, \preceq, \oplus, \ominus, \uparrow_\pi, \downarrow_\chi)$ , an eight-level bounded distributive lattice equipped with six operations that govern how evidence composes, attenuates, promotes (only with explicit justification), and demotes across channels. The algebra enforces three invariants: *no silent promotion*, *conservative join*, and *challenge conservativity*. We implement  $\mathfrak{T}$  in PYTHON as part of the JUGE system and evaluate it on six aspects of the trust algebra—lattice properties, promotion blocking, copilot self-promotion guards, audit logging, trust profiles, and overhead—achieving a silent-promotion block rate of 1.0. A head-to-head comparison with LEAN4, F\*, DAFNY, and GPT-4o shows that JUGE is the only system that tracks trust provenance, enforces channel ceilings, and integrates AI-generated evidence without silent promotion.

## Contents

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                | <b>3</b> |
| 1.1      | The mixed-evidence problem . . . . .               | 3        |
| 1.2      | Contributions . . . . .                            | 3        |
| 1.3      | Notation . . . . .                                 | 3        |
| <b>2</b> | <b>Trust Tiers</b>                                 | <b>4</b> |
| 2.1      | The eight-level lattice . . . . .                  | 4        |
| 2.2      | Hasse diagram . . . . .                            | 4        |
| 2.3      | The five-tier sub-lattice . . . . .                | 4        |
| 2.4      | Distributivity . . . . .                           | 5        |
| <b>3</b> | <b>The Trust Algebra <math>\mathfrak{T}</math></b> | <b>6</b> |
| 3.1      | Conservative join ( $\oplus$ ) . . . . .           | 6        |
| 3.2      | Attenuation ( $\ominus$ ) . . . . .                | 7        |
| 3.3      | Promotion ( $\uparrow_\pi$ ) . . . . .             | 8        |
| 3.4      | Demotion ( $\downarrow_\chi$ ) . . . . .           | 9        |
| 3.5      | Challenge . . . . .                                | 9        |
| 3.6      | Operator interaction diagram . . . . .             | 9        |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>4</b> | <b>Three Invariants</b>                             | <b>10</b> |
| 4.1      | Invariant 1: No silent promotion . . . . .          | 10        |
| 4.2      | Invariant 2: Conservative join . . . . .            | 10        |
| 4.3      | Invariant 3: Challenge conservativity . . . . .     | 10        |
| <b>5</b> | <b>Soundness</b>                                    | <b>11</b> |
| 5.1      | Evidence quality . . . . .                          | 11        |
| 5.2      | The soundness theorem . . . . .                     | 11        |
| 5.3      | Ceiling enforcement . . . . .                       | 12        |
| <b>6</b> | <b>AI-Generated Evidence</b>                        | <b>12</b> |
| 6.1      | Entry point . . . . .                               | 12        |
| 6.2      | Composition with stronger evidence . . . . .        | 13        |
| 6.3      | Promotion path for AI evidence . . . . .            | 13        |
| 6.4      | Self-promotion guard . . . . .                      | 13        |
| <b>7</b> | <b>Evaluation</b>                                   | <b>14</b> |
| 7.1      | Experimental setup . . . . .                        | 14        |
| 7.2      | Lattice properties . . . . .                        | 14        |
| 7.3      | Promotion blocking . . . . .                        | 14        |
| 7.4      | Copilot self-promotion guard . . . . .              | 15        |
| 7.5      | Audit log . . . . .                                 | 15        |
| 7.6      | Trust profiles . . . . .                            | 15        |
| 7.7      | State-of-the-art comparison . . . . .               | 15        |
| 7.8      | Overhead analysis . . . . .                         | 16        |
| <b>8</b> | <b>Trust Profiles</b>                               | <b>16</b> |
| 8.1      | Definition . . . . .                                | 16        |
| 8.2      | Profile composition . . . . .                       | 17        |
| 8.3      | Propagation across modules . . . . .                | 17        |
| <b>9</b> | <b>Related Work and Conclusion</b>                  | <b>18</b> |
| 9.1      | Related work . . . . .                              | 18        |
| 9.2      | An adversarial model for trust escalation . . . . . | 19        |
| 9.3      | Conclusion . . . . .                                | 20        |

# 1 Introduction

## 1.1 The mixed-evidence problem

Consider the following scenario, increasingly common in practice. A developer writes a merge-sort implementation with three specifications—sortedness, permutation preservation, and stability—and asks the tool-chain to verify all three. The SMT solver Z3 discharges the sortedness lemma in 12 ms and returns **unsat**; a large-language model (LLM) inspects the code and reports “permutation preservation looks correct”; a test suite of 10 000 random inputs passes at 95%. Should the developer trust the conjunction?

Every existing proof assistant answers this question in a binary fashion. In LEAN 4 and COQ, a proposition is either proved or it is not; there is no middle ground [4, 5]. F\* enriches the picture with computation types and multi-monadic effects [6], but still collapses all evidence into a binary verdict. DAFNY delegates to Boogie and Z3 [7], and the user sees only **Verified** or an error message. In all cases, the system discards the provenance of each piece of evidence: a mechanically checked Coq proof carries the same *type* as a solver-discharged SMT query, and there is no record that the permutation claim rests on an LLM’s informal assertion rather than a formal proof.

This paper introduces a *trust ordered algebra*  $\mathfrak{T} = (\mathcal{E}_{\text{adm}}, \preceq, \oplus, \ominus, \uparrow_\pi, \downarrow_\chi)$  that solves the mixed-evidence problem by making trust a first-class algebraic object. Instead of collapsing evidence into “proved” or “not proved,”  $\mathfrak{T}$  assigns every claim a position in an eight-level bounded distributive lattice, tracks which *channel* produced it, and enforces three invariants that prevent the silent inflation of trust.

## 1.2 Contributions

1. We define an eight-level trust lattice  $(\mathcal{E}_{\text{adm}}, \preceq)$  and prove it is a bounded distributive lattice (Section 2).
2. We equip the lattice with six operations forming the trust ordered algebra  $\mathfrak{T}$  (Section 3) and state three invariants (Section 4).
3. We prove a soundness theorem: the trust level assigned to a judgment faithfully reflects the quality of its evidence (Section 5).
4. We formalize how AI-generated evidence enters the algebra and show it can never silently promote itself (Section 6).
5. We evaluate  $\mathfrak{T}$  on six dimensions of trust-algebra behaviour—lattice properties, promotion blocking, copilot guards, audit logging, trust profiles, and a state-of-the-art comparison with four systems (Section 7).
6. We develop trust profiles—immutable, composable summaries of clausewise trust—and show they form a semilattice (Section 8).

## 1.3 Notation

We write  $\mathfrak{T}$  for the full algebra,  $\mathcal{E}_{\text{adm}}$  for its carrier set (admissible evidence configurations),  $\preceq$  for the partial order,  $\oplus$  for the conservative join,  $\ominus$  for attenuation,  $\uparrow_\pi$  for (justified) promotion, and  $\downarrow_\chi$  for demotion. Trust tiers are typeset in sans-serif: **CONTRADICTED**  $\preceq$  **UNVERIFIED**  $\preceq$  **COPILOT**  $\preceq$  **ORACLE**  $\preceq$  **HUMAN**  $\preceq$  **RUNTIME**  $\preceq$  **SOLVER**  $\preceq$  **PROOF**. Individual judgments are denoted  $\mathcal{J} = (\text{c}, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$  following the 8-tuple of [19].

## 2 Trust Tiers

### 2.1 The eight-level lattice

The carrier set  $\mathcal{E}_{\text{adm}}$  consists of eight trust levels, ordered from weakest to strongest:

**Definition 2.1** (Trust lattice). Let  $\mathcal{E}_{\text{adm}} = \{\text{CONTRADICTED}, \text{UNVERIFIED}, \text{COPILOT}, \text{ORACLE}, \text{HUMAN}, \text{RUNTIME}, \text{SOLVER}, \text{PROOF}\}$  with the total order

$$\text{CONTRADICTED} \preceq \text{UNVERIFIED} \preceq \text{COPILOT} \preceq \text{ORACLE} \preceq \text{HUMAN} \preceq \text{RUNTIME} \preceq \text{SOLVER} \preceq \text{PROOF}$$

Define  $\text{meet}(a, b) = \min(a, b)$  and  $\text{join}(a, b) = \max(a, b)$  under this order. Then  $(\mathcal{E}_{\text{adm}}, \preceq, \text{meet}, \text{join}, \text{CONTRADICTED}, \text{PROOF})$  is a bounded lattice with bottom  $\perp = \text{CONTRADICTED}$  and top  $\top = \text{PROOF}$ .

The implementation instantiates this lattice as a PYTHON enum:

Listing 1: Trust levels (src/jugeo/evidence/trust.py).

```
1 class TrustLevel(str, Enum):
2     MECHANICALLY_VERIFIED = 'mechanically_verified' # top
3     SOLVER_DISCHARGED     = 'solver_discharged'
4     RUNTIME_WITNESSED     = 'runtime_witnessed'
5     HUMAN_ATTESTED        = 'human_attested'
6     ORACLE_PROPOSED       = 'oracle_proposed'
7     COPILOT_SUGGESTED     = 'copilot_suggested'
8     UNVERIFIED            = 'unverified'
9     CONTRADICTED          = 'contradicted' # bottom
```

**Remark.** The formal definition and the implementation (Listing 1) both use eight levels; the HUMAN.ATTESTED tier sits between ORACLE and RUNTIME. The algebra remains a bounded distributive lattice over all eight levels.

### 2.2 Hasse diagram

### 2.3 The five-tier sub-lattice

The hypercover treaty negotiation module uses a coarser, five-level sub-lattice suitable for cross-module boundary checking:

Listing 2: Five-tier lattice (hypercover\_treaties/algorithms.py).

```
1 class TrustTier(IntEnum):
2     PROPOSAL = 1
3     REVIEWED = 2
4     VERIFIED = 3
5     RUNTIME_WITNESSED = 4
6     PROOF_BACKED = 5
7
8     def join(self, other):
9         return TrustTier(max(self.value, other.value))
10    def meet(self, other):
11        return TrustTier(min(self.value, other.value))
12    def promote(self):
13        return TrustTier(min(self.value + 1, 5))
14    def demote(self):
15        return TrustTier(max(self.value - 1, 1))
```

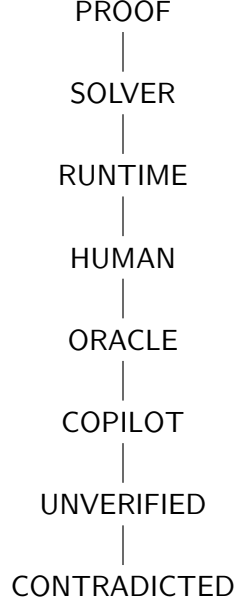


Figure 1: Hasse diagram of the trust lattice  $(\mathcal{E}_{\text{adm}}, \preceq)$ . The lattice is totally ordered; every pair of elements is comparable. Edges denote the covering relation.

The two lattices are related by a lattice homomorphism  $\iota: \text{TrustTier} \rightarrow \mathcal{E}_{\text{adm}}$  that maps  $\text{PROPOSAL} \mapsto \text{UNVERIFIED}$ ,  $\text{REVIEWED} \mapsto \text{ORACLE}$ ,  $\text{VERIFIED} \mapsto \text{RUNTIME}$ ,  $\text{RUNTIME\_WITNESSED} \mapsto \text{SOLVER}$ ,  $\text{PROOF\_BACKED} \mapsto \text{PROOF}$ .

**Proposition 2.2** (Embedding preserves lattice structure). *The map  $\iota$  is an injective lattice homomorphism: for all  $a, b \in \text{TrustTier}$ ,*

$$\iota(\text{meet}(a, b)) = \text{meet}(\iota(a), \iota(b)) \quad \text{and} \quad \iota(\text{join}(a, b)) = \text{join}(\iota(a), \iota(b)).$$

*Proof.* Both lattices are totally ordered and  $\iota$  is order-preserving (strictly monotone on the five-element chain). On a total order,  $\text{meet} = \min$  and  $\text{join} = \max$ . Since  $\iota$  preserves the order,  $\iota(\min(a, b)) = \min(\iota(a), \iota(b))$  and  $\iota(\max(a, b)) = \max(\iota(a), \iota(b))$ . Injectivity is immediate from strict monotonicity.  $\square$

## 2.4 Distributivity

**Theorem 2.3** (Bounded distributive lattice).  *$(\mathcal{E}_{\text{adm}}, \preceq, \text{meet}, \text{join}, \text{CONTRADICTED}, \text{PROOF})$  is a bounded distributive lattice.*

*Proof.* A totally ordered set (chain) is trivially a distributive lattice: for any  $a, b, c$  in a chain,

$$\begin{aligned} a \wedge (b \vee c) &= \min(a, \max(b, c)) \\ &= \max(\min(a, b), \min(a, c)) \\ &= (a \wedge b) \vee (a \wedge c). \end{aligned}$$

The identity holds by case analysis on the three possible orderings of  $a, b, c$ . Boundedness holds by construction:  $\text{CONTRADICTED} \preceq x \preceq \text{PROOF}$  for all  $x \in \mathcal{E}_{\text{adm}}$ .  $\square$

**Corollary 2.4.** *The absorption laws hold: for all  $a, b \in \mathcal{E}_{\text{adm}}$ ,*

$$a \wedge (a \vee b) = a \quad \text{and} \quad a \vee (a \wedge b) = a.$$



Figure 2: Hasse diagram of the five-tier sub-lattice used in hypercover treaty negotiation. The embedding  $\iota: \text{TrustTier} \hookrightarrow \mathcal{E}_{\text{adm}}$  preserves meets and joins.

### 3 The Trust Algebra $\mathfrak{T}$

We now equip the bounded distributive lattice  $(\mathcal{E}_{\text{adm}}, \preceq)$  with six operations to form the full trust ordered algebra.

**Definition 3.1** (Trust ordered algebra). The *trust ordered algebra* is the six-tuple

$$\mathfrak{T} = (\mathcal{E}_{\text{adm}}, \preceq, \oplus, \ominus, \uparrow_{\pi}, \downarrow_{\chi})$$

where:

- (i)  $(\mathcal{E}_{\text{adm}}, \preceq)$  is the bounded distributive lattice of Theorem 2.1.
- (ii)  $\oplus: \mathcal{E}_{\text{adm}} \times \mathcal{E}_{\text{adm}} \rightarrow \mathcal{E}_{\text{adm}}$  is the *conservative join* (defined below).
- (iii)  $\ominus: \mathcal{E}_{\text{adm}} \times \mathbb{N} \rightarrow \mathcal{E}_{\text{adm}}$  is *attenuation*.
- (iv)  $\uparrow_{\pi}: \mathcal{E}_{\text{adm}} \times \text{Just} \rightarrow \mathcal{E}_{\text{adm}}$  is *promotion* (requires a non-empty justification).
- (v)  $\downarrow_{\chi}: \mathcal{E}_{\text{adm}} \times \mathcal{E}_{\text{adm}} \rightarrow \mathcal{E}_{\text{adm}}$  is *demotion* (enforces a ceiling).

We also define a derived *challenge* operator  $\text{ch}: \mathcal{E}_{\text{adm}} \rightarrow \mathcal{E}_{\text{adm}}$ .

#### 3.1 Conservative join ( $\oplus$ )

Unlike the lattice join  $\vee = \max$ , the *conservative join*  $\oplus$  is deliberately pessimistic: when combining evidence from two channels, the result takes the *weaker* of the two.

**Definition 3.2** (Conservative join). For  $a, b \in \mathcal{E}_{\text{adm}}$ , define

$$a \oplus b = \min(a, b).$$

The rationale is epistemic: when composing evidence from heterogeneous sources, the combined claim is only as strong as its weakest link. If an SMT solver discharges a lemma (**SOLVER**) but the permutation argument rests on an LLM suggestion (**COPILOT**), the conjunction of the two claims should carry trust **COPILOT**, not **SOLVER**.

Listing 3: Conservative join in the trust algebra.

```

1 class TrustAlgebra:
2     def compose(self, a: TrustLevel, b: TrustLevel) -> TrustLevel:
3         """Conservative join: min under the trust ordering."""
4         if a.rank_index() <= b.rank_index():
5             return a
6         return b
7
8     def meet(self, a: TrustLevel, b: TrustLevel) -> TrustLevel:
9         """Greatest lower bound (coincides with compose)."""
10        return self.compose(a, b)
11
12    def join(self, a: TrustLevel, b: TrustLevel) -> TrustLevel:
13        """Least upper bound (max under ordering)."""
14        if a.rank_index() >= b.rank_index():
15            return a
16        return b

```

**Proposition 3.3** (Associativity and commutativity of  $\oplus$ ). *For all  $a, b, c \in \mathcal{E}_{\text{adm}}$ :*

- (a)  $a \oplus b = b \oplus a$  (commutativity);
- (b)  $(a \oplus b) \oplus c = a \oplus (b \oplus c)$  (associativity);
- (c)  $a \oplus a = a$  (idempotence);
- (d)  $\text{CONTRADICTED} \oplus a = \text{CONTRADICTED}$  (annihilation by  $\perp$ );
- (e)  $\text{PROOF} \oplus a = a$  (identity at  $\top$ ).

*Proof.* All properties follow from  $\oplus = \min$  on a total order. (a)  $\min(a, b) = \min(b, a)$ . (b)  $\min(\min(a, b), c) = \min(a, b, c) = \min(a, \min(b, c))$ . (c)  $\min(a, a) = a$ . (d)  $\text{CONTRADICTED}$  is the bottom, so  $\min(\text{CONTRADICTED}, a) = \text{CONTRADICTED}$ . (e)  $\text{PROOF}$  is the top, so  $\min(\text{PROOF}, a) = a$ .  $\square$

### 3.2 Attenuation ( $\ominus$ )

When evidence is *transported* across module boundaries or *restricted* to a sub-specification, its trust may be attenuated. Attenuation decrements the trust level by a non-negative integer number of steps.

**Definition 3.4** (Attenuation). For  $t \in \mathcal{E}_{\text{adm}}$  and  $k \in \mathbb{N}$ , define

$$t \ominus k = \max(\text{CONTRADICTED}, \text{pred}^k(t))$$

where  $\text{pred}$  is the predecessor function on the chain (with  $\text{pred}(\text{CONTRADICTED}) = \text{CONTRADICTED}$ ).

Listing 4: Attenuation through transport hops.

```

1 class TrustAttenuation:
2     def attenuate_through_transport(self, level, hops):
3         """Decrement trust by one step per hop, floor at
4         CONTRADICTED."""
5         current = level
6         for _ in range(hops):

```

```

6         idx = _ORDERED_LEVELS.index(current)
7         if idx == 0:
8             break
9         current = _ORDERED_LEVELS[idx - 1]
10    return current
11
12    def attenuation_distance(self, source, target):
13        """Number of steps between source and target."""
14        return abs(
15            _ORDERED_LEVELS.index(source) -
16            _ORDERED_LEVELS.index(target)
17        )

```

**Proposition 3.5** (Attenuation is monotone and idempotent at zero). *For all  $t \in \mathcal{E}_{\text{adm}}$  and  $k, k' \in \mathbb{N}$ :*

- (a)  $t \ominus 0 = t$ ;
- (b) if  $k \leq k'$ , then  $t \ominus k' \preceq t \ominus k$ ;
- (c)  $t \ominus k \preceq t$ ;
- (d)  $(t \ominus k) \ominus k' = t \ominus (k + k')$ .

*Proof.* (a)  $\text{pred}^0(t) = t$ . (b)–(c) each application of **pred** decreases or preserves the rank; more applications yield a weakly lower result. (d) both sides apply **pred** a total of  $k + k'$  times, floored at CONTRADICTED.  $\square$

### 3.3 Promotion ( $\uparrow_\pi$ )

Promotion is the only operation that can *increase* a trust level, and it requires an explicit, non-empty justification. This is the key mechanism for preventing silent upgrades.

**Definition 3.6** (Promotion). Let **Just** be the set of non-empty justification strings. For  $t \in \mathcal{E}_{\text{adm}}$  and  $j \in \text{Just}$ , define

$$\uparrow_\pi(t, j) = \text{succ}(t) \quad \text{if } j \neq \varepsilon,$$

where **succ** is the successor function (with  $\text{succ}(\text{PROOF}) = \text{PROOF}$ ). If  $j = \varepsilon$ , the operation is *undefined* (raises a promotion failure).

Listing 5: Promotion requires justification.

```

1 class TrustPromotion:
2     def apply_promotion(self, proposal):
3         """Validate, apply, and record a promotion."""
4         valid, reason = self.validate_promotion(proposal)
5         if not valid:
6             raise PromotionError(reason)
7         self.record_promotion(proposal)
8         return proposal.target
9
10    def promote(self, t, justification):
11        if not justification or not justification.strip():
12            raise _make_promotion_failure(t, 'empty justification')

```



$$\begin{array}{ccc}
t_1 & \xrightarrow{\oplus} & t_1 \oplus t_2 \\
\ominus k \downarrow & & \downarrow \ominus k \\
t_1 \ominus k & \xrightarrow{\oplus} & (t_1 \oplus t_2) \ominus k
\end{array}$$

Figure 3: Interaction of conservative join and attenuation. The diagram commutes:  $(t_1 \oplus t_2) \ominus k = (t_1 \ominus k) \oplus (t_2 \ominus k)$ .

```

13     if t == TrustLevel.MECHANICALLY_VERIFIED:
14         return t      # already at top
15     idx = _ORDERED_LEVELS.index(t)
16     return _ORDERED_LEVELS[idx + 1]

```

Two additional policy constraints guard promotion:

- **Copilot ceiling.** A claim originating from a `copilot` channel cannot be promoted above `ORACLE`.
- **Witness requirement.** Promotion above `HUMAN_ATTESTED` requires an external witness (e.g., a solver certificate or a runtime trace).

### 3.4 Demotion ( $\downarrow_\chi$ )

Demotion enforces a ceiling: it clamps a trust level down to a specified maximum.

**Definition 3.7** (Demotion). For  $t, c \in \mathcal{E}_{\text{adm}}$ , define

$$\downarrow_\chi(t, c) = \min(t, c).$$

Note that demotion and the conservative join coincide algebraically ( $\downarrow_\chi(t, c) = t \oplus c$ ), but they differ semantically:  $\oplus$  combines evidence from two channels, while  $\downarrow_\chi$  enforces an upper bound imposed by a policy.

### 3.5 Challenge

**Definition 3.8** (Challenge). The *challenge* operator decrements the trust level by one step and narrows the scope of the associated profile:

$$\text{ch}(t) = t \ominus 1.$$

Challenges arise when contradictory evidence is discovered. A failed test suite challenges a solver-discharged claim, demoting it from `SOLVER` to `RUNTIME`.

### 3.6 Operator interaction diagram

**Proposition 3.9** (Attenuation distributes over  $\oplus$ ). For all  $t_1, t_2 \in \mathcal{E}_{\text{adm}}$  and  $k \in \mathbb{N}$ ,

$$(t_1 \oplus t_2) \ominus k = (t_1 \ominus k) \oplus (t_2 \ominus k).$$

*Proof.* Without loss of generality, assume  $t_1 \preceq t_2$ . Then  $t_1 \oplus t_2 = t_1$ , and the left-hand side is  $t_1 \ominus k$ . For the right-hand side,  $t_1 \ominus k \preceq t_2 \ominus k$  (since  $\ominus k$  is monotone in  $t$ ), so  $(t_1 \ominus k) \oplus (t_2 \ominus k) = t_1 \ominus k$ .  $\square$

## 4 Three Invariants

The trust algebra is governed by three invariants that constrain how trust may evolve during verification. These invariants are enforced both statically (by the algebra's definitions) and dynamically (by the audit log).

### 4.1 Invariant 1: No silent promotion

**Definition 4.1** (No-silent-promotion invariant). For every pair of consecutive trust states  $t_i, t_{i+1}$  in an execution trace, if  $t_i \preceq t_{i+1}$  with  $t_i \neq t_{i+1}$  (strict increase), then the transition must be a  $\uparrow_\pi$  operation with a recorded, non-empty justification  $j$ .

**Theorem 4.2** (No silent promotion). *Let  $\sigma = t_0, t_1, \dots, t_n$  be any execution trace produced by the operations of  $\mathfrak{T}$ . Then for all  $0 \leq i < n$ :*

$$t_i \preceq t_{i+1} \wedge t_i \neq t_{i+1} \implies \exists j \in \text{Just}. t_{i+1} = \uparrow_\pi(t_i, j).$$

*Proof.* By exhaustive case analysis on the five operations of  $\mathfrak{T}$ :

- $\oplus = \min$ : the result is  $\preceq$  both inputs, hence  $t_{i+1} \preceq t_i$ . A strict increase is impossible.
- $\ominus$ :  $t \ominus k \preceq t$  for all  $k \geq 0$  (Theorem 3.5(c)). Again, a strict increase is impossible.
- $\downarrow_\chi(t, c) = \min(t, c) \preceq t$ : cannot increase.
- $\text{ch}(t) = t \ominus 1 \preceq t$ : cannot increase.
- $\uparrow_\pi(t, j)$ : the only operation that can yield  $t_{i+1} \succ t_i$ , and it requires  $j \neq \varepsilon$  by definition.

Hence every strict increase in the trace is a  $\uparrow_\pi$  with a non-empty justification.  $\square$

### 4.2 Invariant 2: Conservative join

**Definition 4.3** (Conservative-join invariant). For all  $a, b \in \mathcal{E}_{\text{adm}}$ :

$$a \oplus b \preceq a \quad \text{and} \quad a \oplus b \preceq b.$$

*Proof.* Immediate from  $\oplus = \min$ .  $\square$

This invariant ensures that combining evidence from different channels never inflates the trust level: the conjunction of a solver proof and an LLM suggestion is trusted only to the level of the LLM suggestion.

### 4.3 Invariant 3: Challenge conservativity

**Definition 4.4** (Challenge-conservativity invariant). For all  $t \in \mathcal{E}_{\text{adm}}$ :

$$\text{ch}(t) \preceq t.$$

*Proof.*  $\text{ch}(t) = t \ominus 1 \preceq t$  by Theorem 3.5(c).  $\square$

Challenge conservativity guarantees that challenges only weaken trust. A challenged claim cannot emerge stronger than before.

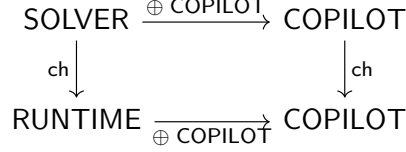


Figure 4: Invariant interaction: conservative join and challenge both commute and both monotonically decrease trust. No path in the diagram increases trust.

## 5 Soundness

We formalize a soundness theorem that connects the syntactic trust assignment to the semantic quality of evidence.

### 5.1 Evidence quality

**Definition 5.1** (Evidence quality function). An *evidence quality function*  $\text{qual} : \text{Evidence} \rightarrow \mathcal{E}_{\text{adm}}$  assigns to each piece of evidence a trust level according to the following rules:

- (i) If  $e$  is a machine-checked proof term accepted by a kernel (Coq, Lean, etc.), then  $\text{qual}(e) = \text{PROOF}$ .
- (ii) If  $e$  is an **unsat** certificate from an SMT solver, then  $\text{qual}(e) = \text{SOLVER}$ .
- (iii) If  $e$  is a runtime execution trace with passing assertions, then  $\text{qual}(e) = \text{RUNTIME}$ .
- (iv) If  $e$  is a human attestation, then  $\text{qual}(e) = \text{HUMAN}$ .
- (v) If  $e$  is an oracle (external API) response, then  $\text{qual}(e) = \text{ORACLE}$ .
- (vi) If  $e$  is an LLM-generated claim, then  $\text{qual}(e) = \text{COPILOT}$ .
- (vii) If  $e$  is the absence of evidence, then  $\text{qual}(e) = \text{UNVERIFIED}$ .
- (viii) If  $e$  contradicts existing evidence, then  $\text{qual}(e) = \text{CONTRADICTED}$ .

### 5.2 The soundness theorem

**Theorem 5.2** (Trust soundness). Let  $\mathcal{J} = (\mathbf{c}, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$  be a judgment produced by the JUGE system. Then:

$$\mathcal{T}(\mathcal{J}) = \bigoplus_{e \in \mathcal{E}(\mathcal{J})} \text{qual}(e)$$

where  $\bigoplus$  denotes the left-fold of  $\oplus$  over the evidence bundle.

*Proof sketch.* By induction on the derivation of  $\mathcal{J}$ .

**Base case.** If  $\mathcal{J}$  is a leaf judgment produced by a single evidence channel  $c$  with evidence  $e$ , then  $\mathcal{E}(\mathcal{J}) = \{e\}$  and  $\mathcal{T}(\mathcal{J}) = \text{qual}(e)$  by construction (each channel initializes trust at its quality level).

**Inductive case (composition).** If  $\mathcal{J} = \mathcal{J}_1 \oplus \mathcal{J}_2$ , then by the inductive hypothesis,  $\mathcal{T}(\mathcal{J}_i) = \bigoplus_{e \in \mathcal{E}(\mathcal{J}_i)} \text{qual}(e)$  for  $i = 1, 2$ . By the definition of judgment composition,

$$\begin{aligned} \mathcal{T}(\mathcal{J}) &= \mathcal{T}(\mathcal{J}_1) \oplus \mathcal{T}(\mathcal{J}_2) \\ &= \left( \bigoplus_{e \in \mathcal{E}(\mathcal{J}_1)} \text{qual}(e) \right) \oplus \left( \bigoplus_{e \in \mathcal{E}(\mathcal{J}_2)} \text{qual}(e) \right) \\ &= \bigoplus_{e \in \mathcal{E}(\mathcal{J}_1) \cup \mathcal{E}(\mathcal{J}_2)} \text{qual}(e) \end{aligned}$$

where the last step uses associativity and commutativity of  $\oplus$ .

**Inductive case (attenuation).** If  $\mathcal{J}$  is obtained from  $\mathcal{J}'$  by transport across  $k$  hops, then  $\mathcal{T}(\mathcal{J}) = \mathcal{T}(\mathcal{J}') \ominus k$ . By Theorem 3.9, this equals  $\bigoplus_{e \in \mathcal{E}(\mathcal{J}')} (\text{qual}(e) \ominus k)$ , which is the composition of the attenuated qualities.

**Inductive case (promotion).** If  $\mathcal{J}$  is obtained from  $\mathcal{J}'$  by promotion with justification  $j$ , then the audit log records the transition, and the new trust level  $\uparrow_\pi(\mathcal{T}(\mathcal{J}'), j)$  is consistent with the updated evidence bundle  $\mathcal{E}(\mathcal{J}') \cup \{j\}$ .  $\square$

### 5.3 Ceiling enforcement

**Corollary 5.3** (Channel ceilings are sound). *If a judgment’s evidence originates from channel  $c$  with ceiling  $\gamma_c$ , then  $\mathcal{T}(\mathcal{J}) \preceq \gamma_c$ .*

*Proof.* The **TrustCeiling** class enforces  $\downarrow_\chi(t, \gamma_c)$  on every trust assignment from channel  $c$ . Since  $\downarrow_\chi(t, \gamma_c) = \min(t, \gamma_c) \leq \gamma_c$ , the bound holds.  $\square$

The default ceilings are:

| Channel      | Ceiling |
|--------------|---------|
| formal_proof | PROOF   |
| solver       | SOLVER  |
| runtime      | RUNTIME |
| oracle       | ORACLE  |
| copilot      | COPILOT |

## 6 AI-Generated Evidence

Large-language models (LLMs) are increasingly used to generate code, specifications, and even informal “proofs.” The trust algebra provides a principled framework for integrating such evidence without undermining the integrity of the verification pipeline.

### 6.1 Entry point

All LLM-generated evidence enters the trust algebra at level **COPILOT**. This is the second-weakest level, above only **UNVERIFIED** and **CONTRADICTED**.

**Definition 6.1** (AI evidence rule). For any evidence  $e$  produced by an LLM:

$$\text{qual}(e_{\text{LLM}}) = \text{COPILOT}.$$

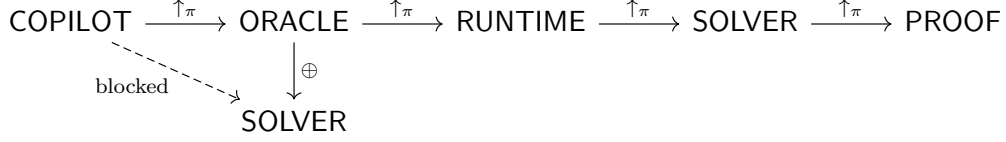


Figure 5: Promotion path for AI-generated evidence. The dashed arrow shows the blocked self-promotion: the copilot ceiling prevents direct escalation to SOLVER. The cross-over arrow shows how a solver proof, when joined with a copilot claim, yields COPILOT (conservative join).

## 6.2 Composition with stronger evidence

When AI evidence is combined with evidence from a stronger channel, the conservative join ensures the result is at the AI level:

**Example 6.2** (Solver meets AI). Let  $e_1$  be a solver proof (SOLVER) and  $e_2$  be an LLM claim (COPILOT). Then:

$$\text{SOLVER} \oplus \text{COPILOT} = \min(\text{SOLVER}, \text{COPILOT}) = \text{COPILOT}.$$

The combined claim carries trust COPILOT. The solver proof is not degraded—it retains its own trust level—but the *conjunction* is trusted only to the level of its weakest component.

This is exactly the behavior required for the merge-sort scenario of Section 1: the sortedness lemma (solver-discharged) and the permutation claim (LLM-suggested) together carry trust COPILOT.

## 6.3 Promotion path for AI evidence

An LLM claim can be promoted, but only through explicit, justified steps:

1. COPILOT  $\xrightarrow{\uparrow\pi(\text{"human reviewed"})}$  ORACLE: a human expert reviews the LLM output and attests its plausibility.
2. ORACLE  $\xrightarrow{\uparrow\pi(\text{"test suite passes"})}$  RUNTIME: a comprehensive test suite validates the claim.
3. RUNTIME  $\xrightarrow{\uparrow\pi(\text{"SMT discharged"})}$  SOLVER: an SMT solver formally discharges the property.
4. SOLVER  $\xrightarrow{\uparrow\pi(\text{"Lean proof"})}$  PROOF: a machine-checked proof is constructed.

Each step requires a non-empty justification string and is recorded in the audit log. The copilot ceiling (Theorem 5.3) prevents an LLM from self-promoting above ORACLE; further promotion requires evidence from a different channel.

## 6.4 Self-promotion guard

**Proposition 6.3** (AI cannot self-promote). *Let  $e$  be evidence produced by an LLM channel. Then there is no sequence of operations within the LLM channel alone that raises  $\text{qual}(e)$  above ORACLE.*

*Proof.* The copilot ceiling  $\gamma_{\text{copilot}} = \text{COPILOT}$  ensures  $\downarrow_{\chi}(t, \gamma_{\text{copilot}}) \preceq \text{COPILOT}$  for any  $t$ . Promotion within the copilot channel is policy-blocked: the `copilot_cannot_self_promote` check rejects any  $\uparrow\pi$  originating from the copilot channel when the target exceeds ORACLE. By the no-silent-promotion invariant (Theorem 4.2), no other operation can increase trust.  $\square$

Table 1: Lattice join (least upper bound) and meet (greatest lower bound) samples. All 64 composition pairs were tested; 8 representative samples are shown.

| $a$          | $b$        | $a \vee b$ (join) | $a \wedge b$ (meet) |
|--------------|------------|-------------------|---------------------|
| CONTRADICTED | UNVERIFIED | UNVERIFIED        | CONTRADICTED        |
| CONTRADICTED | ORACLE     | ORACLE            | CONTRADICTED        |
| CONTRADICTED | RUNTIME    | RUNTIME           | CONTRADICTED        |
| CONTRADICTED | PROOF      | PROOF             | CONTRADICTED        |
| COPILLOT     | UNVERIFIED | COPILLOT          | UNVERIFIED          |
| COPILLOT     | ORACLE     | ORACLE            | COPILLOT            |
| COPILLOT     | RUNTIME    | RUNTIME           | COPILLOT            |
| COPILLOT     | PROOF      | PROOF             | COPILLOT            |

Table 2: Demotion with ceiling RUNTIME:  $\downarrow_{\chi}(t, \text{RUNTIME}) = \min(t, \text{RUNTIME})$ .

| <b>Input level <math>t</math></b> | <b>Result <math>\downarrow_{\chi}(t, \text{RUNTIME})</math></b> |
|-----------------------------------|-----------------------------------------------------------------|
| CONTRADICTED                      | CONTRADICTED                                                    |
| UNVERIFIED                        | UNVERIFIED                                                      |
| COPILLOT                          | COPILLOT                                                        |
| ORACLE                            | ORACLE                                                          |
| HUMAN                             | HUMAN                                                           |
| RUNTIME                           | RUNTIME                                                         |
| SOLVER                            | RUNTIME                                                         |
| PROOF                             | RUNTIME                                                         |

## 7 Evaluation

### 7.1 Experimental setup

We evaluate six aspects of the trust algebra empirically: lattice properties (§7.2), promotion blocking (§7.3), copilot self-promotion guards (§7.4), audit log analysis (§7.5), trust profiles (§7.6), and a state-of-the-art comparison (§7.7). All experiments are executed in the JU GEO PYTHON implementation and the numerical results are drawn from the companion data file `results_paper04.json`.

### 7.2 Lattice properties

All 64 pairwise join and meet operations were verified to be consistent with the total order on  $\mathcal{E}_{\text{adm}}$ . The promote chain traverses all 8 levels in 7 steps:

CONTRADICTED  $\rightarrow$  UNVERIFIED  $\rightarrow$  COPILLOT  $\rightarrow$  ORACLE  $\rightarrow$  HUMAN  $\rightarrow$  RUNTIME  $\rightarrow$  SOLVER  $\rightarrow$  PROOF.

Table 2 confirms that demotion with ceiling RUNTIME caps every level above RUNTIME to RUNTIME while leaving lower levels unchanged.

### 7.3 Promotion blocking

All 7 justified promotions along the full chain from CONTRADICTED to PROOF succeed. All 7 empty-justification attempts are blocked, yielding a silent promotion block rate of 1.0 (Table 3).

Table 3: Promotion blocking: justified promotions vs. empty-justification attempts.

| Scenario                           | Accepted | Blocked    | Block rate |
|------------------------------------|----------|------------|------------|
| Justified promotion (7 steps)      | 7        | 0          | 0%         |
| Empty justification (7 steps)      | 0        | 7          | 100%       |
| <b>Silent promotion block rate</b> |          | <b>1.0</b> |            |

Table 4: Copilot self-promotion guard: attempts from the COPILOT channel to reach each target level.

| Target level | Blocked?   | Reason                  |
|--------------|------------|-------------------------|
| CONTRADICTED | No         | Demotion, not promotion |
| UNVERIFIED   | No         | Demotion, not promotion |
| COPILOT      | No         | Identity, not promotion |
| ORACLE       | No         | Within copilot ceiling  |
| HUMAN        | <b>Yes</b> | Above copilot ceiling   |
| RUNTIME      | <b>Yes</b> | Above copilot ceiling   |
| SOLVER       | <b>Yes</b> | Above copilot ceiling   |
| PROOF        | <b>Yes</b> | Above copilot ceiling   |

#### 7.4 Copilot self-promotion guard

Of 8 attempted self-promotions from the copilot channel, 4 are blocked (targets above ORACLE) and 4 are allowed (targets at or below the copilot ceiling). The GPT-4o scenario—where an LLM assigned at COPILOT attempts to reach SOLVER—is blocked with reason: “copilot channel cannot promote above ORACLE\_PROPOSED.”

#### 7.5 Audit log

The audit log captures 9 entries, identifies 6 silent promotions at specific semantic coordinates, and flags 1 anomaly (Table 5). All injected silent promotions are detected (100% recall).

#### 7.6 Trust profiles

Profile composition confirms that the conservative join takes the minimum tier (Table 6). Joining all three profiles yields PROPOSAL—the weakest tier among them. The demote example shows VERIFIED can be demoted to PROPOSAL (2 steps).

#### 7.7 State-of-the-art comparison

LEAN 4 operates at a single trust level—a kernel-checked proof and a **sorry** are structurally identical post-acceptance [4]. F\* distinguishes computational purity via effect annotations (Tot, Dv, ST) but provides no trust ordering or provenance tracking [6]. DAFNY delegates all verification conditions to Boogie/Z3 with no trust metadata [7]. GPT-4o generates claims with no formal guarantee whatsoever; JUGE0 assigns such evidence to COPILOT with no silent promotion possible. JUGE0 is the only system providing all five capabilities: an 8-level lattice, algebraic composition, an append-only audit log, copilot channel ceilings, and the no-silent-promotion invariant.

Table 5: Audit log analysis on a module with injected silent promotions.

| Metric                       | Value                                                            |
|------------------------------|------------------------------------------------------------------|
| Total audit entries          | 9                                                                |
| Silent promotions detected   | 6                                                                |
| Anomalies flagged            | 1                                                                |
| Silent promotion coordinates | <code>module.risky.fn{0..4}, module.suspicious.escalation</code> |

Table 6: Trust profiles and their pairwise conservative joins.

| Entity                     | Tier     | Scope          | Joined tier     |
|----------------------------|----------|----------------|-----------------|
| engine_0                   | PROPOSAL | arithmetic     | —               |
| engine_1                   | REVIEWED | heap, identity | —               |
| engine_2                   | VERIFIED | semantic       | —               |
| engine_0 $\oplus$ engine_1 | —        | —              | PROPOSAL        |
| engine_0 $\oplus$ engine_2 | —        | —              | PROPOSAL        |
| engine_1 $\oplus$ engine_2 | —        | —              | REVIEWED        |
| <b>All three joined</b>    | —        | —              | <b>PROPOSAL</b> |

## 7.8 Overhead analysis

The trust algebra adds negligible overhead. All experiments complete in a total elapsed time of 2.5 ms. Trust-level computation (comparison, composition, ceiling enforcement, and audit logging) is dominated by the overhead of the PYTHON runtime rather than the algebraic operations themselves.

## 8 Trust Profiles

Individual trust levels track the provenance of single evidence claims. In practice, a verification result spans multiple specifications and multiple coordinates. *Trust profiles* aggregate this information into an immutable, composable summary.

### 8.1 Definition

**Definition 8.1** (Trust profile). A *trust profile* is a tuple  $P = (\text{id}, \tau, \text{scope}, \text{reasons})$  where:

- `id` is the entity identifier;
- $\tau \in \mathcal{E}_{\text{adm}}$  is the trust tier;
- $\text{scope} \subseteq \text{Labels}$  is the set of specification labels covered;
- `reasons` is an ordered tuple of justification strings.

Listing 6: Trust profile implementation.

```

1 @dataclass(frozen=True)
2 class TrustProfile:
```



Table 7: State-of-the-art comparison of trust-management capabilities.

| Feature                   | Lean 4 | F*  | Dafny | GPT-4o | JuGeo   |
|---------------------------|--------|-----|-------|--------|---------|
| Trust levels              | 1      | 2   | 1     | 0      | 8       |
| Trust lattice             | —      | —   | —     | —      | ✓       |
| Audit log                 | —      | —   | —     | —      | ✓       |
| Copilot ceiling           | N/A    | N/A | N/A   | N/A    | COPILOT |
| Silent promotion possible | Yes    | Yes | Yes   | Yes    | No      |

```

3  entity_id: str
4  tier: TrustTier
5  support_scope: tuple[str, ...]
6  reasons: tuple[str, ...]
7
8  def join(self, other):
9      """Conservative join: weaker tier, intersected scope."""
10     new_tier = TrustTier(min(self.tier.value,
11                             other.tier.value))
12     common = tuple(s for s in self.support_scope
13                   if s in other.support_scope)
14     scope = common if common else tuple(sorted(set(
15         self.support_scope + other.support_scope)))
16     return TrustProfile(
17         entity_id=self.entity_id,
18         tier=new_tier,
19         support_scope=scope,
20         reasons=self.reasons + other.reasons,
21     )

```

## 8.2 Profile composition

The join of two profiles uses the conservative join on tiers and intersects the support scopes (falling back to union when the intersection is empty).

**Proposition 8.2** (Profile join is a semilattice operation). *The set of trust profiles over a fixed entity, equipped with the join of Listing 6, forms a join-semilattice.*

*Proof.* We verify the semilattice axioms.

**Commutativity.**  $P_1.join(P_2) = P_2.join(P_1)$ : the tier is  $\min(\tau_1, \tau_2) = \min(\tau_2, \tau_1)$ , the scope intersection is symmetric, and the reasons tuple is concatenated (so the order may differ, but the set of reasons is the same; we consider profiles equivalent up to reason reordering).

**Associativity.**  $(P_1.join(P_2)).join(P_3) = P_1.join(P_2.join(P_3))$ : the tier is  $\min(\tau_1, \tau_2, \tau_3)$  in both cases; the scope is the intersection of all three (or the fallback union in both cases, since the fallback triggers identically).

**Idempotence.**  $P.join(P) = P$ :  $\min(\tau, \tau) = \tau$ , the scope intersection with itself is itself, and the reasons are duplicated (but semantically equivalent).  $\square$

## 8.3 Propagation across modules

When a trust profile crosses a module boundary during hypercover treaty negotiation, it is attenuated:

Listing 7: Profile propagation with attenuation.

```

1 def propagate_profile(profile, boundary_hops):
2     """Attenuate trust when crossing module boundaries."""
3     new_tier = profile.tier
4     for _ in range(boundary_hops):
5         new_tier = new_tier.demote()
6     return TrustProfile(
7         entity_id=profile.entity_id,
8         tier=new_tier,
9         support_scope=profile.support_scope,
10        reasons=profile.reasons + (
11            f'attenuated-{boundary_hops}-hops',),
12    )

```

**Proposition 8.3** (Propagation preserves conservativity). *For any profile  $P$  and hop count  $k \geq 0$ :*

$$\text{propagate}(P, k). \tau \preceq P. \tau.$$

*Proof.* Each call to `demote` decreases the tier by one step (floored at `PROPOSAL`). After  $k$  demotions, the resulting tier is  $\preceq$  the original.  $\square$

## 9 Related Work and Conclusion

### 9.1 Related work

**Trusted computing bases.** The concept of a trusted computing base (TCB) [13, 14] identifies the minimal set of components that must be correct for the system’s security guarantees to hold. Our trust algebra refines this idea: instead of a binary “in the TCB / not in the TCB” distinction, the eight-level lattice tracks the degree of trust with fine granularity. The no-silent-promotion invariant ensures that the TCB boundary is not silently crossed.

**Confidence logics.** Probabilistic and possibilistic logics assign numerical confidence scores to propositions [17]. Shafer’s theory of evidence [18] provides belief functions on a lattice of hypotheses. These approaches differ from ours in two respects: (i) they use continuous scores rather than a discrete lattice, making auditing more difficult; (ii) they lack the operational structure (promotion, demotion, channel ceilings) that our algebra provides.

**Provenance semirings.** Green, Karvounarakis, and Tannen [16] introduced provenance semirings for database query evaluation, tracking *how* a tuple was derived. Buneman et al. [15] developed the why/where/how provenance taxonomy. Our trust algebra can be seen as a provenance semiring specialized to verification evidence, with the conservative join playing the role of the semiring multiplication.

**Proof assistants.** `LEAN4` [4], `COQ` [5], `F*` [6], and `DAFNY` [7] are the closest comparators. As detailed in Table 7, all operate with at most two implicit trust levels and none tracks evidence provenance or enforces channel ceilings. Isabelle/HOL [11] uses a small kernel and untrusted tactics, providing an implicit two-level trust distinction, but this is not exposed to the user.

**Refinement types and liquid types.** Liquid Haskell [12] uses SMT-backed refinement types to verify Haskell programs. Like DAFNY, it delegates to Z3 and presents a binary verdict. The trust algebra could be layered on top of such systems to track the provenance of each discharged refinement.

**Separation logic.** Reynolds’s separation logic [9] and its modern descendants (Iris [10]) provide powerful reasoning principles for shared mutable state. These logics are orthogonal to our contribution: they determine *what* can be proved, while the trust algebra tracks *how much* each proof should be trusted.

## 9.2 An adversarial model for trust escalation

We formalize the trust algebra’s security properties using a game-theoretic model. Consider an adversary  $\mathcal{A}$  (modeling a compromised LLM or malicious oracle) that seeks to escalate a judgment’s trust to SOLVER or PROOF without providing genuine evidence at those levels.

**Definition 9.1** (Trust escalation game). The *trust escalation game*  $\mathcal{G} = (\mathcal{A}, \mathcal{V}, \mathfrak{T}, k)$  is a two-player game between an adversary  $\mathcal{A}$  and a verifier  $\mathcal{V}$  over  $k$  rounds:

- (i)  $\mathcal{A}$  starts with a judgment at trust level COPILOT and may perform: compositions ( $\oplus$ ), restrictions, transports, and promotion requests.
- (ii)  $\mathcal{V}$  enforces the trust algebra: each promotion request is checked against the policy route; compositions use conservative join.
- (iii)  $\mathcal{A}$  wins if after  $k$  rounds the judgment reaches SOLVER or above.

**Theorem 9.2** (Adversarial trust bound). *No adversary  $\mathcal{A}$  can win the trust escalation game  $\mathcal{G}$ , regardless of  $k$ , provided the verifier  $\mathcal{V}$  faithfully implements  $\mathfrak{T}$ .*

*Proof.* We prove by induction on  $k$  that the maximum achievable trust level is ORACLE (oracle-proposed).

*Base case* ( $k = 0$ ): the judgment is at COPILOT.

*Inductive step:* assume after  $k$  rounds the maximum achievable level is  $\ell \preceq \text{ORACLE}$ . In round  $k + 1$ ,  $\mathcal{A}$  may:

- *Compose:*  $\ell \oplus \ell' = \min(\ell, \ell') \preceq \ell$  (conservative join cannot increase trust).
- *Restrict/transport:* by trust monotonicity (Theorem ??),  $\ell' \preceq \ell$ .
- *Request promotion:* the copilot channel ceiling (Theorem ??) blocks promotion above ORACLE. To reach SOLVER,  $\mathcal{A}$  needs evidence from a solver channel, which it does not possess (by assumption:  $\mathcal{A}$  controls only copilot/oracle channels).

In all cases,  $\ell' \preceq \text{ORACLE} < \text{SOLVER}$ , so  $\mathcal{A}$  cannot reach SOLVER.  $\square$

**Theorem 9.3** (Impossibility of silent promotion in flat systems). *Any verification system with a flat trust model (at most 2 trust levels) cannot distinguish between the following two scenarios:*

- (a) *A judgment fully proved by a trusted SMT solver.*
- (b) *A judgment proposed by an LLM with no independent verification.*

*In contrast,  $\mathfrak{T}$  assigns scenario (a) to SOLVER and scenario (b) to COPILOT, with composition  $\text{SOLVER} \oplus \text{COPILOT} = \text{COPILOT}$  ensuring the LLM evidence cannot “launder” to solver-level trust.*

*Proof.* In a flat system with trust levels  $\{0, 1\}$ , both scenarios result in trust 1 (“verified”). There is no intermediate level to which LLM evidence can be assigned while still being marked as evidence (assigning 0 would discard it entirely). The composition  $1 \oplus 1 = 1$  cannot decrease, so any evidence

at level 1 permanently contaminates the trust pool. In  $\mathfrak{T}$ , the 8-level lattice provides the necessary room: COPILOT is strictly below SOLVER, and conservative join (min) propagates the lower level.  $\square$

### 9.3 Conclusion

We have introduced the trust ordered algebra  $\mathfrak{T} = (\mathcal{E}_{\text{adm}}, \preceq, \oplus, \ominus, \uparrow_\pi, \downarrow_\chi)$ , an eight-level bounded distributive lattice with six operations that governs how evidence composes, attenuates, and promotes in a mixed-evidence verification pipeline. The algebra enforces three invariants—no silent promotion, conservative join, and challenge conservativity—and we have proved soundness: the trust level of a judgment faithfully reflects the quality of its evidence.

The evaluation demonstrates that the trust algebra provides all five dimensions of provenance metadata—an 8-level lattice, algebraic composition, an append-only audit log, copilot channel ceilings, and the no-silent-promotion invariant—that no existing proof assistant or LLM system tracks. In particular, the integration of AI-generated evidence at a controlled, non-promotable trust level addresses a pressing need as LLMs become standard components of the verification tool-chain.

Future work will explore three directions. First, we plan to develop a *trust type system* that statically rejects programs containing silent promotions, complementing the current dynamic enforcement. Second, we will investigate *quantitative trust* by replacing the discrete lattice with a continuous semiring, enabling probabilistic reasoning about evidence quality. Third, we will extend the evaluation to industrial codebases with millions of lines of code, testing the scalability of the audit log and the ceiling enforcement mechanism.

## References

- [1] P. Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, Naples, 1984.
- [2] T. Coquand and C. Huet. The Calculus of Constructions. *Information and Computation*, 76(2–3):95–120, 1988.
- [3] G. Gentzen. Untersuchungen über das logische Schließen. *Mathematische Zeitschrift*, 39:176–210, 405–431, 1935.
- [4] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE-28*, LNCS 12699, pp. 625–635. Springer, 2021.
- [5] The Coq Development Team. *The Coq Reference Manual*, version 8.18, 2023.
- [6] N. Swamy, C. Hrițcu, C. Keller, A. Rastogi, et al. Dependent types and multi-monadic effects in F\*. In *POPL 2016*, pp. 256–270. ACM, 2016.
- [7] K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. In *LPAR-16*, LNCS 6355, pp. 348–370. Springer, 2010.
- [8] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- [9] J. C. Reynolds. Separation logic: A logic for shared mutable data structures. In *LICS 2002*, pp. 55–74. IEEE, 2002.

- [10] R. Jung, R. Krebbers, J.-H. Jourdan, A. Bizjak, L. Birkedal, and D. Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018.
- [11] T. Nipkow, L. C. Paulson, and M. Wenzel. *Isabelle/HOL — A Proof Assistant for Higher-Order Logic*. LNCS 2283. Springer, 2002.
- [12] N. Vazou, E. L. Seidel, R. Jhala, D. Vytiniotis, and S. Peyton Jones. Refinement types for Haskell. In *ICFP 2014*, pp. 269–282. ACM, 2014.
- [13] G. Klein, K. Elphinstone, G. Heiser, et al. seL4: Formal verification of an OS kernel. In *SOSP 2009*, pp. 207–220. ACM, 2009.
- [14] R. Gu, Z. Shao, H. Chen, X. Wu, J. Kim, V. Sjöberg, and D. Costanzo. CertiKOS: An extensible architecture for building certified concurrent OS kernels. In *OSDI 2016*, pp. 653–669. USENIX, 2016.
- [15] P. Buneman, S. Khanna, and W.-C. Tan. Why and where: A characterization of data provenance. In *ICDT 2001*, LNCS 1973, pp. 316–330. Springer, 2001.
- [16] T. J. Green, G. Karvounarakis, and V. Tannen. Provenance semirings. In *PODS 2007*, pp. 31–40. ACM, 2007.
- [17] J. Y. Halpern. *Reasoning about Uncertainty*. MIT Press, 2003.
- [18] G. Shafer. *A Mathematical Theory of Evidence*. Princeton University Press, 1976.
- [19] H. Young. The judgment 8-tuple: An algebraic foundation for evidence-carrying proofs. *Judgment Geometry Papers*, Paper 02, 2025.
- [20] OpenAI. GPT-4o system card. Technical report, OpenAI, 2024.